hashlock.

2024

Security Audit

Kingdomly 2 (Bridge)

September 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

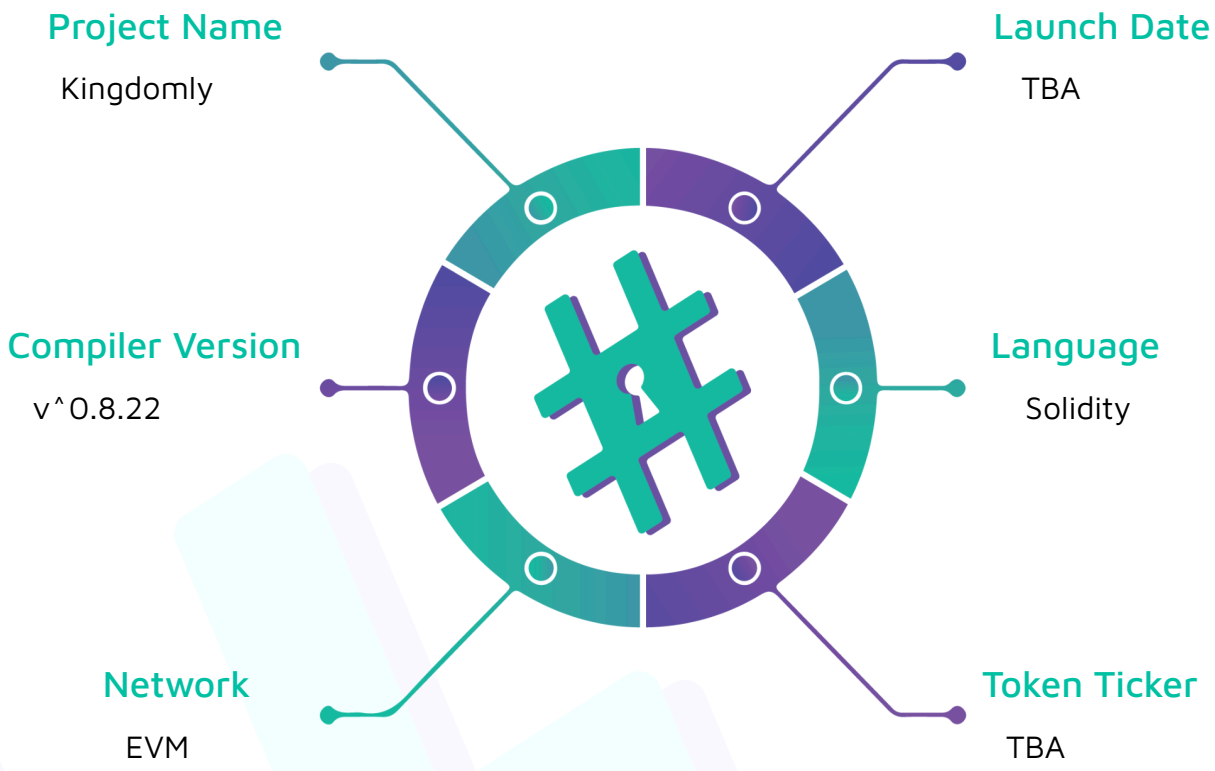
The Kingdomly team partnered with Hashlock to conduct a security audit of their MasterContractV3_ONFT.sol, MasterContractV3_Proxy.sol, and MasterContractV3.sol smart contracts. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

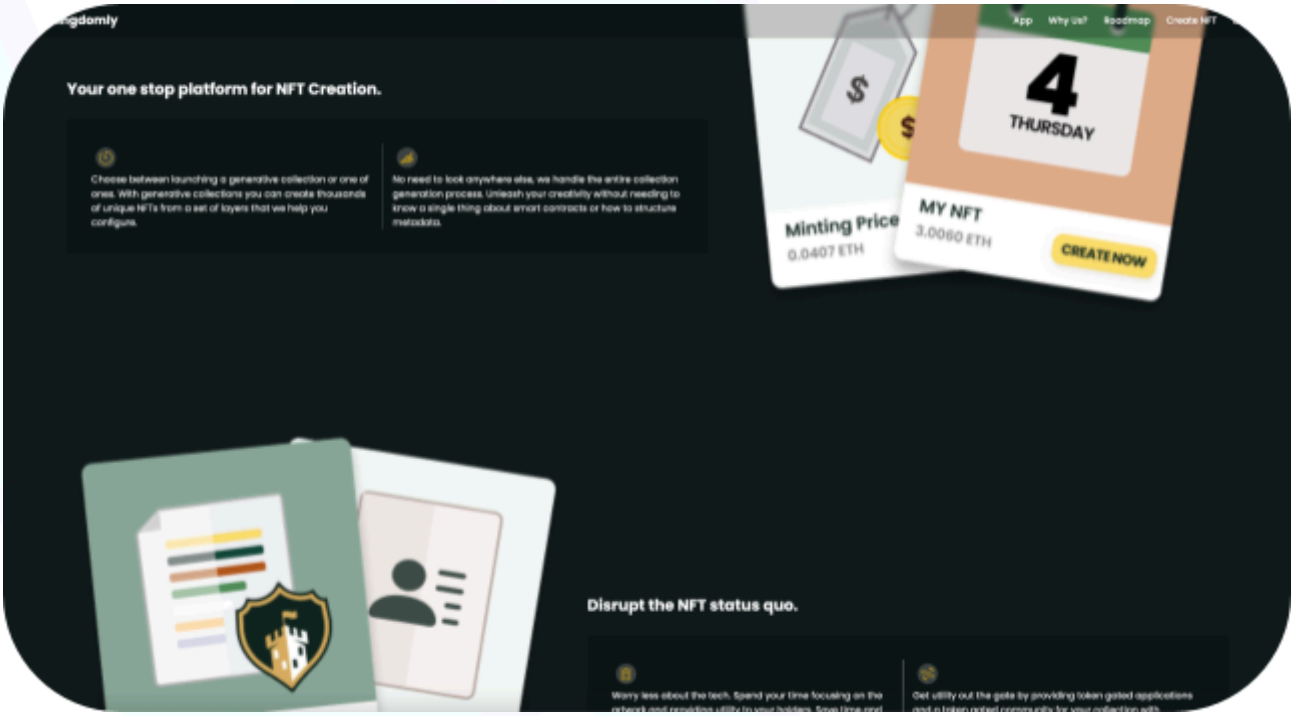
Kingdomly is a platform that enables users to create and manage NFT collections with ease. It provides tools for launching generative or unique NFTs without requiring technical skills. The platform handles the entire NFT creation process, including smart contracts and metadata, and offers features like customizable mint pages, private sales, and token-gated applications. Kingdomly aims to simplify NFT creation, allowing creators to focus on their art and community engagement.

Project Name: Kingdomly
Compiler Version: ^0.8.22
Website: https://www.kingdomly.info/
Logo:

Visualised Context:



Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Kingdomly project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Kingdomly Smart Contracts**
Platform	**EVM / Solidity**
Audit Date	**September, 2024**
Contract 1	MasterContractV3_ONFT.sol
Contract 1 MD5 Hash	83612914e8df7d124f738ceffae1bcfa
Contract 2	MasterContractV3_Proxy.sol
Contract 2 MD5 Hash	6bf84f5ad0cb5d01e291238d250a687a
Contract 3	MasterContractV3.sol
Contract 3 MD5 Hash	3bde51729f1764b6c86b44f1715d1d74

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

1 High-severity vulnerability

2 Medium-severity vulnerabilities

1 Low-severity vulnerability

2 QAs

1 Gas Optimisation

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour

MasterContractV3_ONFT.sol
- Allows users to:
  - Sends/BatchSends tokens cross-chain
- Allows admins to:
  - Set a baseURI
  - Set a new kingdomly fee contract
	Contract achieves this functionality.

MasterContractV3_Proxy.sol
- Allows users to:
  - Sends/BatchSends tokens cross-chain
- Allows admins to:
  - Set a baseURI
  - Set a new kingdomly fee contract
	Contract achieves this functionality.

MasterContractV3.sol
- Allows users to:
  - BatchMint/DelegateMint
- Allows admins to:
  - Change the max mint per mint group
  - Set a mint quota
  - Control the presale status
  - Set the maximum number of tokens that can be minted in a batch
  - Set the sale price
  - Airdrop NFTs
  - Change the minting status
  - Set the base URI
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the Kingdomly project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Kingdomly project smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01]MasterContractV3_ONFT#batchSend, MasterContractV3_Proxy#batchSend - Lack of a check of the native coin amount sent

Description

The `batchSend` function calculates the total message fee through the `quoteBatchSend` function but does not check if the amount of native coins sent from the caller is enough for the total native fee.

Vulnerability Details

The `batchSend` function is missing a check if the `msg.value` is equal to the total native fee.
It allows users to execute the functions without paying fees.

Impact

Users can execute cross-chain transfers without paying fees.

Recommendation

Add a check to make sure that `msg.value` is equal to the total native fee.

Status

Resolved

Medium

[M-01]MasterContractV3_ONFT#getBridgeNativeFee, MasterContractV3_Proxy#getBridgeNativeFee - Division Before Multiplication results in precision loss

Description

The `getBridgeNativeFee` function calculates the native fee by first dividing `getOneDollarInWei` by 100 and then multiplying by `bridgeFeeInCents`.

Impact

Division Before Multiplication is not a good solidity design pattern and it can result in precision loss.

Recommendation

It's recommended that the native fee be calculated by first multiplying `getOneDollarInWei` by `bridgeFeeInCents` and then dividing by 100.

Status

Resolved

[M-02] MasterContractV3 - Incorrect value of the threeDollarsEth variable due to the Eth price changes

Description

The `threeDollarsEth` variable is designed to store the Eth amount equivalent to $3. However, the variable is only set in the constructor and the Eth price is changed in real time so the `threeDollarsEth` variable will have an incorrect value in the future.

Impact

Users will pay the incorrect fee.

Recommendation

Calculating the current ETH price and getting the correct amount of ETH equivalent to $3 is recommended.

Status

Resolved

Low

[L-01] MasterContractV3#setAffiliatePercentage, setMaxMintPerWallet, stopOrStartpresaleMint, MasterContractV3_ONFT#setNewKingdomlyFeeContract, MasterContractV3_Proxy#setNewKingdomlyFeeContract - Lack of emitting events

Description

The `setAffiliatePercentage`, `setMaxMintPerWallet`, `stopOrStartpresaleMint` and `setNewKingdomlyFeeContract` functions are missing events when key storages are updated.

Recommendation

Add relevant events based on the variables to be updated.

Status

Resolved

QA

[Q-01]MasterContractV3_ONFT#setNewKingdomlyFeeContract, MasterContractV3_Proxy#setNewKingdomlyFeeContract - Missing zero address checks

Description

The `setNewKingdomlyFeeContract` functions are missing zero address checks for the parameter values from input.

Recommendation

Add checks to make sure that the new fee contract to be set is not a zero address.

Status

Resolved

[Q-02] MasterContractV3#_batchMint - Unnecessary assignment

Description

The local variable `totalCost` is updated at the end of the function even if it's not used after the assignment.

```
function _batchMint(
    address delegatedCaller,
    uint256 amount,
    uint256 mintId,
    address affiliate
) internal returns (uint256) {
        . . .
        if (
          affiliatePercentage != 0 &&
          affiliate != address(0) &&
          affiliate != msg.sender &&
          affiliate != delegatedCaller
        ) {
```

```
            . . .
            totalCost -= affiliateAmount;
        }
        . . .
}
```

Recommendation

Remove updating `totalCost` variable at the end of `if` condition.

Status

Resolved

Gas

[G-01]MasterContractV3_ONFT#batchSend, MasterContractV3_Proxy#batchSend - _sendParams.length is read multiple times

Description

The `batchSend` function excessively reads `_sendParams.length` value from calldata on each loop iteration.

Recommendation

Consider caching it in a local variable instead and 2 for loops could be combined.

Status

Resolved

Centralisation

The Kingdomly project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the Kingdomly project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.